

Reset detachment in the V1 GLIF model: investigation, derivation, and results

All experiments in this folder ran on 2026-08-31 with the Euler membrane step that was active at the time. Sections 2--3 derive the gradient math for that step. Section 11 redoes the derivation for the exact membrane propagator introduced on 2026-09-25 (commit [b0f4cbf](#)), under which these experiments have **not** been re-run.

Companion documents in this folder:

File	Content
literature_review.md	Primary-source review of reset detachment
literature_addendum.md	Controlled studies, reset-aware and exact-gradient counterexamples
asc_gradient_literature.md	Whether the spike-to-ASC path should be detached
experimental_conclusion.md	Decision note: membrane reset (Section 9.3 here)
asc_experimental_conclusion.md	Decision note: ASC path (Section 9.4 here)
surrogate_reset_factorial_conclusion.md	Decision note: reset × surrogate amplitude (Sections 9.5--9.9 here)
run_*.log , benchmark_*.json	Training runs: per-update loss history, step time, memory
gradient_*.log	Single-update raw gradient diagnostics

1. Why this investigation started

The original TensorFlow GLIF state update propagated gradients through the previous spike, `prev_z`, in both the voltage reset and the after-spike current (ASC) update:

```
new_asc = asc_decay * asc + expand_dims(prev_z, -1) * asc_amps
new_v = decay * v + current_factor * c1 - prev_z
```

The CUDA state-transition kernel was first aligned with this fully differentiable update. That restored the reset and ASC gradients through `prev_z` and removed the backward-only dampening of the voltage self-loop.

This version was correct relative to the active TensorFlow equations, but a full-model comparison showed a small performance cost: approximately 140 MiB more TensorFlow GPU memory and about 2.5% more runtime per steady-state step. Part of that difference came from a custom-gradient shape-safety expression that unnecessarily evaluated `zeros_like(output) + gradient` even when the gradient already had the correct shape. The wrapper was corrected to return compatible gradients directly and broadcast only scalar or genuinely incompatible cotangents.

That optimization addressed unnecessary TensorFlow operations, but it did not answer the scientific question: should the voltage reset be differentiable at all? And, separately: should the spike-triggered ASC increment be differentiable?

2. Model and notation (Euler step used in the experiments)

2.1 Forward dynamics

Voltages are normalized per neuron type so that $E_L = 0$ and $V_{th} = 1$. The time step is $\Delta t = 1$ ms. For neuron i at step t (the neuron index is dropped where unambiguous):

$$\begin{aligned} a_{t+1,j} &= \rho_j a_{t,j} + A_j z_t, & j &\in \{1, 2\} \\ x_{t+1,b} &= \sigma_b x_{t,b} + \frac{\varepsilon}{\tau_b} I_{t,b}, & b &= 1, \dots, B \\ y_{t+1,b} &= \sigma_b y_{t,b} + \Delta t \sigma_b x_{t,b} \\ v_{t+1} &= \alpha v_t + \kappa \left(\sum_b y_{t,b} + \sum_j a_{t,j} \right) - z_t \\ r_{t+1} &= \max(r_t + z_t T^{\text{ref}} - 1, 0) \\ z_{t+1} &= H(v_{t+1} - 1) \cdot \mathbf{1}[r_{t+1} = 0] \end{aligned}$$

with the synaptic drive

$$I_{t,b}^i = \sum_k W_{ik} \beta_{s(ik),b} z_{t-d_{ik}}^k + I_{t,b}^{i,\text{LGN}} + I_{t,b}^{i,\text{BKG}}.$$

Symbol	Meaning	Code name
v_t	normalized membrane potential	<code>v</code>
z_t	binary spike	<code>prev_z / new_z</code>
$a_{t,j}$	after-spike current j	<code>asc</code>
$x_{t,b}, y_{t,b}$	rise and current of the alpha-shaped PSC for basis receptor b ($B = 4$)	<code>psc_rise, psc</code>
r_t	refractory counter (integer steps)	<code>r</code>
$\alpha = e^{-\Delta t/\tau_m}, \tau_m = C_m/g$	membrane decay per step	<code>decay</code>
$\kappa = (1 - \alpha)/g$	current-to-voltage factor	<code>current_factor</code>
$\rho_j = e^{-k_j \Delta t}$	ASC decay per step	<code>asc_decay</code>
$A_j = \text{asc_amps}_j / (V_{th} - E_L)$	normalized ASC jump	<code>asc_amps</code>
$\sigma_b = e^{-\Delta t/\tau_b}$	synaptic basis decay	<code>syn_decay</code>

Symbol	Meaning	Code name
$W_{ik}, \beta_{s,b}, d_{ik}$	recurrent weight, basis weight of synapse type s , delay	<code>recurrent_weight_values</code> , <code>synaptic_basis_weights</code> , <code>delays</code>
T^{ref}	refractory period in steps, $\lceil t_{\text{ref}}/\Delta t \rceil \geq 1$	<code>t_ref_steps</code>

Two timing details of this Euler step matter below:

- v_{t+1} uses $a_{t,j}$, not $a_{t+1,j}$. The ASC that spike z_t injects therefore first reaches the membrane at v_{t+2} .
- The reset subtracts exactly $V_{\text{th}} - E_L = 1$ (soft, subtractive reset). All experiments used the soft reset (`--hard_reset` is off by default).

Membrane decay values for the full 203,816-neuron network (201 node types, weighted by neuron count; the 10k experiments use a subset of these types):

Quantity	5th pct	Median	95th pct	Range
α	0.892	0.937	0.960	0.833--0.981
$1 - \alpha$	0.040	0.063	0.108	
t_{ref} (ms)	1.85	3.8	6.4	

2.2 Backward rules

The forward pass is identical in every condition studied here. Only these backward rules change:

Surrogate derivative. The default (non-Gaussian) pseudo-derivative is triangular, with peak height $\gamma = \text{dampening_factor}$:

$$\frac{\partial z_t}{\partial v_t} := g_t = \gamma \max(0, 1 - |v_t - 1|) \cdot \mathbf{1}[r_t = 0].$$

Its support is $0 < v_t < 2$. In normalized units this covers the **whole depolarized subthreshold range** from rest to threshold, and an equal range above it. After a soft reset, a neuron that just crossed threshold lands near 0, at the edge of the support. The refractory mask zeroes g_t while $r_t > 0$.

Reset and ASC detach flags. Write the two spike paths with backward weights $\varepsilon_R, \varepsilon_A \in \{0, 1\}$:

$$v_{t+1} = \dots - [\varepsilon_R z_t + (1 - \varepsilon_R) \text{sg}(z_t)], \quad a_{t+1,j} = \dots + A_j [\varepsilon_A z_t + (1 - \varepsilon_A) \text{sg}(z_t)],$$

where `sg` is `stop_gradient`. `--detach_reset` sets $\varepsilon_R = 0$; `--nodetach_reset` sets $\varepsilon_R = 1$. `--detach_asc_reset` sets $\varepsilon_A = 0$; `--nodetach_asc_reset` sets $\varepsilon_A = 1$.

Recurrent dampening. The backward pass of the recurrent synaptic current scales the spike cotangent by $\lambda_{\text{rec}} = \text{recurrent_dampening_factor}$:

$$\left. \frac{\partial \mathcal{L}}{\partial z_t^k} \right|_{\text{rec}} = \lambda_{\text{rec}} \sum_i \sum_b \frac{\partial \mathcal{L}}{\partial I_{t+d_{ik},b}^i} W_{ik} \beta_{s(ik),b}.$$

The forward current and the weight gradient are unaffected.

Detached state. r_t is always detached. With a hard reset, the refractory clamp $v \leftarrow V_{\text{reset}}$ gives zero voltage gradient while clamped.

3. Gradient derivation

3.1 Local Jacobian of one neuron

Take the per-neuron state $s_t = (v_t, a_{t,1}, a_{t,2})$ and hold the synaptic input fixed for now. Using $\partial z_t / \partial v_t = g_t$:

$$J_t = \frac{\partial s_{t+1}}{\partial s_t} = \begin{pmatrix} \alpha - \varepsilon_R g_t & \kappa & \kappa \\ \varepsilon_A A_1 g_t & \rho_1 & 0 \\ \varepsilon_A A_2 g_t & 0 & \rho_2 \end{pmatrix}.$$

BPTT propagates the adjoint $\lambda_t = \partial \mathcal{L} / \partial s_t$ backward as

$$\lambda_t = J_t^\top \lambda_{t+1} + \frac{\partial \mathcal{L}_t}{\partial s_t},$$

so credit from step $t + n$ reaches step t through the product $J_t^\top J_{t+1}^\top \cdots J_{t+n-1}^\top$.

The first column is where the spike enters. Each spike path contributes one term proportional to g_t :

- the reset contributes $-\varepsilon_R g_t$ to $\partial v_{t+1} / \partial v_t$;
- the ASC increment contributes $\varepsilon_A A_j g_t$ to $\partial a_{t+1,j} / \partial v_t$, which reaches the voltage one step later through κ .

3.2 The membrane self-loop

The direct voltage derivative is

$$\frac{\partial v_{t+1}}{\partial v_t} = \begin{cases} \alpha - g_t & \text{attached reset } (\varepsilon_R = 1), \\ \alpha & \text{detached reset } (\varepsilon_R = 0). \end{cases}$$

With the reset detached, the self-loop alone is the passive leak, and $\prod_k \alpha = \alpha^n$ decays geometrically. It cannot explode by itself. Any growth of detached-reset gradients must therefore come from the **closed loops through spikes**: neuron $\rightarrow$ recurrent synapse $\rightarrow$ other neuron $\rightarrow$ spike $\rightarrow$ back, and neuron $\rightarrow$ own ASC $\rightarrow$ own voltage $\rightarrow$ spike.

With the reset attached, $0 \leq g_t \leq \gamma$ gives $\alpha - \gamma \leq \alpha - g_t \leq \alpha$. So $|\alpha - g_t| \leq \alpha$ whenever $\gamma \leq 2\alpha$, which holds for every neuron when $\gamma \leq 1.66$ ($\alpha \geq 0.833$). In that range the attached reset never enlarges the self-loop. It only shrinks it, and for large g_t it can flip its sign.

This explains the initially surprising observation that **detaching the reset makes gradients larger**: the attached reset contributes the negative term $-g_t$, which cancels part of the positive membrane-memory derivative and, more importantly, damps the spike-mediated loops (Section 3.3). Detachment removes that cancellation.

3.3 Spike gain: how many extra spikes a voltage perturbation produces

The self-loop alone cannot explode, so what matters is how strongly a voltage perturbation drives the *spike* variable, which feeds every recurrent loop. Consider a neuron inside the surrogate band with $g_t \approx \bar{g} \leq \gamma$ for

a few membrane time constants, and no other inputs. Perturb its voltage by δv at step t_0 . In the linearized backward model, the perturbation evolves as $\delta v_{t_0+n} = (\partial v_{t+1}/\partial v_t)^n \delta v$, and each step it generates $\delta z_{t_0+n} = \bar{g} \delta v_{t_0+n}$. The total spike perturbation is

$$G \equiv \sum_{n \geq 0} \delta z_{t_0+n} = \begin{cases} \bar{g} \sum_n \alpha^n = \frac{\bar{g}}{1-\alpha} & \text{detached,} \\ \bar{g} \sum_n (\alpha - \bar{g})^n = \frac{\bar{g}}{1-\alpha+\bar{g}} & \text{attached.} \end{cases}$$

The attached estimator satisfies $G < 1$ for every γ , and $G \rightarrow 1/(1 + (1 - \alpha)/\bar{g})$ saturates as γ grows. The detached estimator grows **linearly in γ** without bound.

A natural reference value: for a non-leaky integrator with subtractive reset, one unit of injected charge ($V_{th} - E_L$) produces exactly one extra spike in the long run; leak makes the true value smaller. The attached reset builds this charge conservation into the backward model: the derivative "knows" that a spike removes the charge that caused it. The detached reset lets the same perturbation trigger surrogate spikes again on every subsequent step, until it has leaked away.

Upper bounds ($\bar{g} = \gamma$) using the network's α distribution:

Reset	γ	G at median α	G over 5th--95th pct of α
detached	0.100	1.59	0.92--2.50
detached	0.125	1.99	1.16--3.13
detached	0.150	2.39	1.39--3.75
detached	0.175	2.78	1.62--4.38
detached	1.000	15.9	9.2--25.0
attached	0.100	0.61	0.48--0.71
attached	1.000	0.94	0.90--0.96

The detached estimator matches the reference scale when $\gamma \approx 1 - \alpha$ (about 0.04--0.11 here). At $\gamma = 1$ it overcounts by more than an order of magnitude.

This is a single-neuron linear approximation. It assumes g_t stays roughly constant for $1/(1 - \alpha) \approx 16$ steps, and it ignores the refractory mask and the dependence of g_t on v_t . It is meant to explain the scaling, not to predict exact thresholds.

3.4 Network loops and the stability boundary

In the full network the linearized step is

$$\delta s_{t+1} = M \delta s_t + \sum_d U_d \text{diag}(g_{t-d}) P_v \delta s_{t-d},$$

where M holds the passive decays ($\alpha, \rho_j, \sigma_b, \kappa$), P_v selects voltages, and the columns of U_d say where a spike goes after delay d : $-\varepsilon_R$ on the neuron's own voltage, $\varepsilon_A A_j$ on its own ASCs, and $\lambda_{\text{rec}} W_{ik} \beta_{s,b} e/\tau_b$ on each target's PSC rise.

A cotangent that travels once around a recurrent loop (spike of $k \rightarrow$ PSC of $i \rightarrow$ voltage of $i \rightarrow$ spike of $i \rightarrow$...) is multiplied by roughly

$$\text{loop gain} \approx G \times \lambda_{\text{rec}} \times R,$$

where R is the summed synaptic-to-voltage response of the targets. Consequences:

1. **Detached reset:** $G \propto \gamma$, so the loop gain grows linearly with $\gamma \lambda_{\text{rec}}$. There is a critical amplitude γ^* above which the loop gain exceeds 1 and gradients grow exponentially backward in time.
2. **Attached reset:** G saturates below 1, so raising γ cannot push the loop gain arbitrarily high through this mechanism. That is why attached reset tolerates $\gamma = 1$.
3. **Horizon dependence:** when the per-traversal gain is slightly above 1, the growth compounds with the number of traversals, which scales with sequence length T . A setting can be finite at $T = 100$ and overflow at $T = 500$. Conversely, when the system is contractive, longer sequences do not increase the norm.
4. **FP16 overflow:** FP16's largest finite value is 65,504. A gradient that is merely large in FP32 becomes NaN/Inf in FP16.

3.5 The ASC path

With $\varepsilon_A = 1$, a spike at t adds $A_j \delta z_t$ to $a_{t+1,j}$. That ASC decays with ρ_j and drives the voltage by $\kappa a_{t+n,j}$ on every later step. The ASC path is therefore a second spike-mediated loop: voltage $\rightarrow$ own spike $\rightarrow$ own ASC $\rightarrow$ own voltage. It acts on slow time scales ($1/k_j$ up to hundreds of ms), which is the mechanism that carries spike history forward.

Its sign is set by A_j . In the 203k network, **90.3% of neurons have both ASC amplitudes negative**, and 98.4% have a net negative integrated ASC drive on the voltage ($\sum_j A_j / (g(1 - \rho_j)) < 0$). For almost all neurons the ASC path is therefore a **negative feedback** loop in the linearized backward model, like the attached reset. Keeping it attached adds negative terms to the spike loops; detaching it removes them. This is consistent with the measured *increase* in gradient norm when the ASC path is detached (Section 9.4). It does not prove it, because the complete Jacobian also contains the recurrent paths. The constant- g approximation of Section 3.3 is not reliable on the ASC time scales, so no quantitative ASC gain is given here.

Detaching the ASC increment is not a "smaller reset detachment". It removes a credit pathway through a slow biological state.

3.6 Why voltage-path dampening was removed

An earlier experimental update used

```
dampened_v = straight_through_dampen(v, voltage_gradient_dampening)
new_v = decay * dampened_v + current_factor * c1 - stop_gradient(prev_z)
```

`straight_through_dampen(x, d)` returns $x(1 - d) + \text{sg}(xd)$. Its forward value is x and its derivative is $1 - d$. With $d = 0.5$, the self-loop derivative becomes

$$\frac{\partial v_{t+1}}{\partial v_t} = (1 - d) \alpha = 0.5 \alpha \approx 0.42\text{--}0.49.$$

Credit through the membrane then decays as $(0.5\alpha)^n$ instead of α^n : after 10 steps, $(0.5 \cdot 0.937)^{10} \approx 5 \times 10^{-4}$ instead of $0.937^{10} \approx 0.52$. This is not mild stabilization. It removes membrane-based temporal credit on every step, including steps without spikes. It also does not address the mechanism of Section 3.4, which runs through spikes, not through the passive leak. The final implementation preserves the full voltage self-loop.

4. What the literature supports

The supplied paper, Eshraghian et al., *Training Spiking Neural Networks Using Lessons From Deep Learning* (2023), recommends detaching the reset as a practical surrogate- gradient convention. Its main empirical basis is Zenke and Vogels (2021).

The more precise conclusion from the literature is:

- Reset detachment is a common and defensible default.
- The adverse behavior of differentiable resets is strongly coupled to surrogate derivative scale. With a normalized surrogate, attached and detached resets can perform similarly.
- Explicitly recurrent SNNs are included in the empirical evidence; this is not only a result for layered feedforward SNNs.
- Reset-aware and exact-gradient methods provide counterexamples where propagating reset information is viable, so "attached" is not intrinsically wrong.
- No published controlled benchmark is close enough to this long-horizon, heterogeneous, explicitly recurrent GLIF model to settle the question without a local ablation.
- The literature stabilizes training through surrogate normalization, recurrent-path control, initialization, and gradient clipping. It does not prescribe constant backward-only dampening of the passive voltage self-loop.

On the ASC path: the official LSNN and e-prop implementations keep the spike-to- adaptation increment differentiable, even when they stop spike gradients through other paths. No primary source was found that recommends detaching it during ordinary surrogate-gradient BPTT.

Sources and details: [literature_review.md](#), [literature_addendum.md](#), [asc_gradient_literature.md](#).

5. The final gradient design

The membrane and ASC updates at the time of the experiments were

```
asc_spike = tf.stop_gradient(prev_z) if self._detach_asc_reset else prev_z
new_asc = self.asc_decay * asc + tf.expand_dims(asc_spike, axis=-1) *
self.asc_amps
reset = tf.stop_gradient(prev_z) if self._detach_reset else prev_z
new_v = self.decay * v + self.current_factor * c1 - reset
```

With the recommended defaults (`detach_reset=True`, `detach_asc_reset=False`), the gradient behavior is:

Path	Gradient behavior
<code>v -> new_v</code>	Full derivative <code>decay</code> (α)
<code>c1 -> new_v</code>	Full derivative <code>current_factor</code> (κ)
<code>prev_z -> new_v</code> (reset)	Detached ($\varepsilon_R = 0$)
<code>prev_z -> new_asc</code>	Differentiable through both ASC amplitudes ($\varepsilon_A = 1$)
refractory state	Detached
hard-reset refractory voltage	Zero voltage gradient while clamped

The two spike paths have independent flags, because they are different scientific interventions (Section 3.5):

```
--detach_reset      / --nodetach_reset
--detach_asc_reset  / --nodetach_asc_reset
```

The differentiable CUDA kernel implements the same rule. With the defaults, its `prev_z` gradient contains the two ASC-amplitude contributions and no `-new_v_gradient` reset contribution. Forward CUDA equations are numerically unchanged.

6. Surrogate and recurrent-gradient settings

The project has three distinct controls that should not be conflated:

1. `dampening_factor` (γ) sets the peak of the spike surrogate derivative.
2. `recurrent_dampening_factor` (λ_{rec}) scales gradients returned through the recurrent synaptic current.
3. `voltage_gradient_dampening` was intended to attenuate the passive voltage self-loop.

The wrapper launcher previously defaulted `dampening_factor` to `1`, even though a nearby comment indicated `0.1`. It now defaults to `0.1`, matching `multi_training.py`. The recurrent dampening default remains `0.1`. The voltage-dampening compatibility flag now defaults to `0.0` and does not affect the active state equation.

Section 3.4 shows why γ and the reset mode cannot be chosen independently: the detached reset needs a small γ , and the safe value also depends on λ_{rec} , the precision, and the sequence length.

7. Global-gradient clipping

An optional `--global_clipnorm` training flag was added. Clipping is applied after mixed-precision loss-scale gradients have been unscaled and before `optimizer.apply_gradients`:

```
scaled loss
-> raw scaled gradients
-> mixed-precision unscaling
-> pre-clip global-norm measurement
```

```
-> optional global-norm clipping
-> optimizer update
```

This ordering is necessary because clipping loss-scaled gradients would make the effective threshold depend on the dynamic loss scale.

None gradients retain their positions so they remain aligned with the corresponding variables. When `--debug_gradients` is enabled, training prints the pre-clipping global norm. A value of `--global_clipnorm 0` disables clipping while retaining norm measurement.

Clipping is disabled by default. A threshold learned from a small network should not be assumed safe or useful for the 203k-neuron network.

8. Verification performed

8.1 Behavioral tests

A state-transition test independently observes the two `prev_z` paths:

- the gradient from `new_v` to `prev_z` must be absent or zero when the reset is detached;
- the gradient from `new_asc` to `prev_z` must equal the sum of the two ASC amplitudes when the ASC path is attached.

The same behavior is tested through the TensorFlow and CUDA state-transition backends. CUDA parity coverage also checks FP32, FP16, soft reset, and hard reset.

Gradient-clipping tests verify correct global-norm scaling, preservation of missing gradients, reporting of the raw norm, and no gradient modification when clipping is disabled.

The isolated-GPU CUDA suite first completed with **13 passed**. After the independent ASC flag was added, the state-transition and clipping suite passed all 17 tests, covering the independent reset and ASC paths, FP16/FP32, and soft/hard reset. Python compilation, Ruff lint, and `git diff --check` also passed.

8.2 Real 10k-neuron smoke test

One FP16 training update with detached reset, $\gamma = 0.1$, $\lambda_{\text{rec}} = 0.1$, no voltage dampening and no clipping had finite gradients (pre-clip global norm 0.984 evoked, 0.852 spontaneous; full numbers in Section 9.3). The update took about 18.3 s including first-step graph work, with about 352 MiB TensorFlow peak memory.

The same seed with `--global_clipnorm 0.9` clipped the evoked branch (maximum absolute gradient 0.125 $\rightarrow$ 0.115) and left the spontaneous branch unchanged, because its norm was below 0.9. This verified the training integration. It does **not** establish 0.9 as a recommended full-network threshold.

9. Experiments

9.1 Common protocol

All runs used the 10,000-neuron network from `GLIF_network_n11_full` (2,044,519 recurrent synapses, 40,000 BKG synapses) on an NVIDIA L40S, with:

```
--neurons 10000 --batch_size 2 --grating_batch_size 1 --gray_batch_size 1
--dtype float16 --gradient_checkpointing --gradient_checkpoint_chunk_size
25
--train_recurrent --train_noise --notrain_input
--recurrent_dampening_factor 0.1 --voltage_gradient_dampening 0.0 --
global_clipnorm 0.0
--sync_cost 0 --osi_cost 0 --recurrent_weight_regularization 0
```

- Constant learning rate 0.005, soft reset, triangular surrogate.
- Segmented exact BPTT with chunks of 25 steps.
- Seeds 3000, 3001, 3002, matched across conditions. Forward dynamics are identical across conditions; only ε_R , ε_A and γ change.
- **Simplified screening loss:** firing-rate and voltage terms only. Synchronization, OSI/DSI and recurrent-weight regularization are off.
- Trainable gradients cover two tensors: recurrent weights and BKG weights (2,084,519 elements in total). LGN input weights are frozen.
- Training runs report 1 warm-up update plus 99 measured updates (sequence length 100) or 1 + 24 (sequence length 500). "Curve mean" is the mean loss over all measured updates; "final- k " is the mean of the last k .
- Gradient diagnostics are a single update with `--debug_gradients`, reporting the pre-clip global norm and the largest absolute element for the evoked and spontaneous branches.

Lower loss is better throughout. Paired differences are computed per seed.

9.2 Ten-update pilot: reset detached vs attached, $\gamma = 0.1$

`benchmark_detach_*.json`, `benchmark_nodetach_*.json` (1 + 9 updates).

Seed	Detached	Attached	Detached – attached
3000	7.5452	7.5532	-0.0080
3001	7.1910	7.1891	+0.0019
3002	7.7471	7.7429	+0.0042

No consistent difference after 10 updates. This motivated the 100-update runs.

9.3 Reset detached vs attached, $\gamma = 0.1$, 100 updates

`run_100_detach_*`, `run_100_nodetach_*`, `gradient_detach.log`, `gradient_nodetach.log`.

Seed	Detached end	Attached end	Δ end	Δ curve mean	Δ final-20
3000	7.4603	7.4785	-0.0182	-0.0062	-0.0181
3001	7.1892	7.2139	-0.0247	-0.0027	-0.0178
3002	7.7340	7.7483	-0.0143	+0.0081	-0.0003

Δ = detached – attached. Mean endpoints: 7.4612 vs 7.4802.

Branch	Detached norm	Attached norm	Ratio	Detached max	Attached max
Evoked	0.984	0.575	1.71×	0.1255	0.0266
Spontaneous	0.852	0.482	1.77×	0.0815	0.0282

The detached endpoint was lower in all three seeds, but the effect (about 0.019) is small relative to seed-to-seed spread, one seed was worse over the whole curve, and seed 3002 shows almost no difference over the final 20 updates. This is weak but consistent endpoint evidence. The 1.7× larger norm is the cancellation predicted in Section 3.2; it stayed finite.

9.4 Reset × ASC detachment, $\gamma = 0.1$

```
run_100_{detach,nodetach}[_ascdetach]_*, gradient_{detach,nodetach}[_ascdetach].log.
```

Reset	ASC	Seed 3000	Seed 3001	Seed 3002	Mean end	Mean curve	Mean final-20
detached	attached	7.4603	7.1892	7.7340	7.4612	7.5069	7.4871
attached	attached	7.4785	7.2139	7.7483	7.4802	7.5072	7.4991
detached	detached	7.6141	7.3386	7.8903	7.6143	7.5832	7.6282
attached	detached	7.5375	7.2792	7.8327	7.5498	7.5401	7.5609

Penalty for detaching the ASC path (ASC-detached – ASC-attached, per seed):

Reset	Δ end	Δ curve mean	Δ final-20
detached	+0.154, +0.149, +0.156	+0.080, +0.074, +0.076	+0.145, +0.138, +0.141
attached	+0.059, +0.065, +0.084	+0.026, +0.030, +0.043	+0.049, +0.059, +0.077

Initial gradients, seed 3000:

Reset	ASC	Evoked norm	Spont. norm	Evoked max	Spont. max
detached	attached	0.984	0.852	0.1255	0.0815
attached	attached	0.575	0.482	0.0266	0.0282
detached	detached	1.291	1.095	0.0657	0.0787
attached	detached	0.708	0.592	0.0277	0.0292

Detaching ASC worsened every loss statistic in all six paired comparisons, by an effect several times larger than the reset effect. It also **increased** the global norm in both reset modes, consistent with the negative-feedback sign of the ASC path (Section 3.5). Detached reset with attached ASC was the best condition on every seed.

9.5 Reset × surrogate amplitude, sequence length 100

`run_gamma1_100_{detach,nodetach}_*` against the $\gamma = 0.1$ runs of Section 9.3.

Reset	γ	Seed 3000	Seed 3001	Seed 3002	Mean end	Mean curve	Mean final-20
detached	0.1	7.4603	7.1892	7.7340	7.4612	7.5069	7.4871
attached	0.1	7.4785	7.2139	7.7483	7.4802	7.5072	7.4991
detached	1.0	7.5441	7.2152	7.7567	7.5053	7.5028	7.5009
attached	1.0	7.4706	7.1907	7.7490	7.4701	7.5203	7.4948

Paired differences against detached 0.1:

Comparison	Δ end	Δ curve mean	Δ final-20
attached 1.0 – detached 0.1	+0.010, +0.002, +0.015	+0.014, +0.008, +0.018	+0.010, +0.001, +0.013
detached 1.0 – detached 0.1	+0.084, +0.026, +0.023	+0.022, −0.015, −0.020	+0.054, −0.001, −0.011
attached 1.0 – attached 0.1	−0.008, −0.023, +0.001	+0.008, +0.006, +0.026	−0.009, −0.017, +0.012

Detached 0.1 beat attached 1.0 in every seed, but only by 0.002--0.015 at this short horizon. Attached 1.0 was viable and beat attached 0.1 at the endpoint on two seeds. Detached 1.0 looks merely mediocre here, and even has a lower curve mean on two seeds; Section 9.6 shows this loss curve cannot be trusted.

9.6 Raw gradients versus surrogate amplitude, sequence length 100

`gradient_gamma*_detach*.log, gradient_gamma1_nodetach.log`; seed 3000.

Reset	γ	Precision	Evoked norm	Spont. norm	Evoked max	Spont. max	Finite
attached	0.1	FP16	0.575	0.482	0.0266	0.0282	yes
attached	1.0	FP16	1.051	0.874	0.0451	0.0461	yes
detached	0.100	FP16	0.984	0.852	0.1255	0.0815	yes
detached	0.125	FP16	1.228	1.063	0.2437	0.1614	yes
detached	0.150	FP16	1.573	1.343	0.4743	0.3256	yes
detached	0.175	FP16	2.207	1.754	0.9299	0.5629	yes
detached	0.2 / 0.3 / 0.5 / 1.0	FP16	NaN	not reached	NaN	--	no
detached	1.0	FP32	1.66×10^7	7.08×10^6	9.83×10^6	3.92×10^6	yes

Observations:

- Every NaN run reported 2,084,519 nonfinite elements, which is **every trainable element**. The overflow contaminated the whole backward pass, not a few isolated entries.
- The FP32 repeat of detached 1.0 is finite but about 10^7 times larger than the stable settings. Its largest element exceeds the FP16 maximum (65,504) by a factor of about 150. This is the exponential growth of Section 3.4, not an FP16 artefact.
- Between $\gamma = 0.1$ and 0.175 , the largest evoked element almost exactly doubles per $+0.025$ (ratios 1.94, 1.95, 1.96), i.e. about $e^{27\gamma}$. Extrapolating this trend gives about 1.8 at $\gamma = 0.2$, which is finite, yet the run was NaN. The onset is therefore sharper than the trend. Whether this is an intermediate FP16 overflow or a genuine instability threshold is unresolved, because FP32 was not run at 0.2.
- Attached 1.0 has a norm comparable to detached 0.1 and smaller largest elements: the attached reset holds $\gamma = 1$ in check, as Section 3.3 predicts.

FP16 training uses dynamic loss scaling, which can skip updates whose gradients are nonfinite. The ordinary-looking detached-1.0 loss curve of Section 9.5 is therefore not evidence of successful training.

9.7 Detached surrogate sweep, sequence length 100

`run_gamma{0p125,0p15,0p175}_100_detach_*`.

γ	Seed 3000	Seed 3001	Seed 3002	Mean end	Mean curve	Mean final-20
0.100	7.4603	7.1892	7.7340	7.4612	7.5069	7.4871
0.125	7.4294	7.1506	7.7020	7.4273	7.4965	7.4597
0.150	7.3836	7.1129	7.6667	7.3877	7.4795	7.4274
0.175	7.3578	7.0777	7.6287	7.3547	7.4595	7.3976

Paired differences against $\gamma = 0.1$ (end / final-20):

γ	Seed 3000	Seed 3001	Seed 3002
0.125	-0.031 / -0.024	-0.039 / -0.029	-0.032 / -0.030
0.150	-0.077 / -0.058	-0.076 / -0.062	-0.067 / -0.059
0.175	-0.103 / -0.083	-0.112 / -0.095	-0.105 / -0.091

Larger stable amplitudes improved every loss statistic in every seed, monotonically. At sequence length 100 alone, this suggested 0.1 was conservative. Section 9.8 shows the gain does not transfer to the production horizon.

9.8 Raw gradients at the production horizon, sequence length 500

`gradient_seq500_*.log`; seed 3000.

Candidate	Evoked norm	Spont. norm	Evoked max	Spont. max	Finite
detached, 0.1	0.839	0.823	0.0861	0.0667	yes

Candidate	Evoked norm	Spont. norm	Evoked max	Spont. max	Finite
attached, 1.0	0.845	0.755	0.0326	0.0380	yes
detached, 0.175	NaN	not reached	NaN	--	no (all elements)

- Detached 0.175, finite at $T = 100$, overflows at $T = 500$. The sequence-100 optimum must be rejected. This is the horizon dependence of Section 3.4: the loop gain at 0.175 is high enough to compound over 500 steps.
- Detached 0.1 has a slightly **smaller** norm at $T = 500$ than at $T = 100$ (0.839 vs 0.984), consistent with a contractive regime at this amplitude.
- The stable boundary at $T = 500$ lies somewhere in $0.1 < \gamma < 0.175$. Values in between were not tested.

9.9 Final comparison at sequence length 500

`run_seq500_25_{detach_0p1,attach_1p0}_*` (1 + 24 updates).

Candidate	Seed 3000	Seed 3001	Seed 3002	Mean end	Mean curve	Mean final-10
detached, 0.1	6.6771	6.3999	6.9056	6.6609	6.8062	6.7181
attached, 1.0	6.7009	6.4207	6.9318	6.6845	6.8200	6.7386
Seed	Δ end	Δ curve mean	Δ final-10			
3000	+0.0238	+0.0118	+0.0198			
3001	+0.0209	+0.0126	+0.0180			
3002	+0.0263	+0.0172	+0.0237			

Δ = attached 1.0 – detached 0.1. Detached 0.1 was better on every statistic in every seed. This is the most relevant local result, because it uses the production temporal horizon.

9.10 Runtime and memory

Median step time (s) and TensorFlow peak memory (MiB) per seed:

Condition	T	Median step (3000 / 3001 / 3002)	Peak memory (3000 / 3001 / 3002)
detached 0.1	100	0.222 / 0.222 / 0.221	395 / 398 / 393
attached 0.1	100	0.306 / 0.276 / 0.242	395 / 398 / 394
detached 0.1, ASC detached	100	0.306 / 0.297 / 0.301	400 / 401 / 396
attached 0.1, ASC detached	100	0.298 / 0.302 / 0.297	395 / 390 / 402
detached 0.125	100	0.303 / 0.304 / 0.305	390 / 395 / 398

Condition	T	Median step (3000 / 3001 / 3002)	Peak memory (3000 / 3001 / 3002)
detached 0.15	100	0.303 / 0.296 / 0.300	388 / 394 / 397
detached 0.175	100	0.293 / 0.301 / 0.300	394 / 397 / 392
detached 1.0	100	0.273 / 0.277 / 0.284	396 / 392 / 387
attached 1.0	100	0.283 / 0.283 / 0.295	394 / 397 / 398
detached 0.1	500	1.393 / 1.402 / 1.388	701 / 765 / 786
attached 1.0	500	1.396 / 1.403 / 1.391	687 / 693 / 698

- At $T = 100$, peak memory does not depend on the condition (differences of a few MiB).
- Step times are not controlled. The γ sweep runs the same computation as the detached-0.1 baseline, yet took about 0.30 s instead of 0.22 s. The differences track host load on the shared multi-GPU machine, not the gradient rule.
- At $T = 500$ the step times are equal within noise. Detached 0.1 used about 58 MiB more peak memory on average (751 vs 693 MiB), with large seed-to-seed spread.

The reset choice should be based on optimization behavior, not on these resource numbers.

9.11 How the results line up with the derivation

Prediction (Section 3)	Observation (Section 9)
Detaching the reset removes a cancelling term, so norms grow	1.7× larger norms at $\gamma = 0.1$ (9.3)
Detached spike gain grows linearly in γ ; there is a critical γ^*	Norms rise steeply with γ ; NaN at $\gamma \geq 0.2$ for $T = 100$; FP32 norm about 10^7 at $\gamma = 1$ (9.6)
Attached spike gain saturates, so large γ remains stable	Attached 1.0 finite at $T = 100$ and $T = 500$ (9.6, 9.8)
Marginal loop gain compounds with horizon	Detached 0.175 finite at $T = 100$, NaN at $T = 500$ (9.8)
Contractive regime: longer horizon does not grow the norm	Detached 0.1 norm $0.984 \rightarrow 0.839$ from $T = 100$ to 500 (9.8)
ASC path is mostly negative feedback	Detaching ASC raised the norm in both reset modes (9.4)

The derivation explains *stability*. It does not predict *which stable setting trains faster*. In the spike-gain table, attached 1.0 is closest to the reference value of 1, yet detached 0.1 trained slightly faster at $T = 500$. A plausible reading is that the mild overcount of detached 0.1 strengthens useful credit, but that is an inference that these experiments do not test.

10. Current conclusion

ASC path: keep attached. This is the best-supported result. The literature keeps this path differentiable, it is a negative-feedback credit path through a slow state, and detaching it worsened every loss statistic in all six paired comparisons while increasing the gradient norm.

Membrane reset: detached, with $\gamma = 0.1$. Confidence is moderately high for the tested regime. A detached reset requires a small surrogate amplitude, because without the reset's negative derivative the spike gain grows linearly with γ . Detached 1.0 explodes, and detached 0.175 explodes at the production horizon. An attached reset tolerates $\gamma = 1$, but attached 1.0 trained more slowly than detached 0.1 in every matched production-horizon comparison.

The recommended configuration is:

```
--detach_reset
--nodetach_asc_reset
--dampening_factor 0.1
--recurrent_dampening_factor 0.1
--voltage_gradient_dampening 0.0
```

$\gamma = 0.1$ is not a universal normalized-surrogate constant. It is an empirically safe amplitude for this architecture, precision, sequence length, recurrent dampening and integration scheme. It must be re-validated if any of these change, and it must be tuned at the production sequence length, never at $T = 100$.

10.1 Limitations

- 10k neurons and batch size 2, versus 203k neurons and the production batch.
- Simplified loss: no OSI/DSI, synchronization or recurrent regularization.
- Short training: 100 updates at $T = 100$; 25 at $T = 500$.
- No validation loss and no biological activity metrics.
- Three seeds.
- Euler integration only (Section 11).
- The derivation is a linear, single-neuron approximation of a nonlinear recurrent system. It explains scaling and stability, not exact thresholds.

10.2 Recommended next step

Run a short, matched, full-network pilot with the complete objective: detached 0.1 as the primary configuration, attached 1.0 as the only remaining serious control, ASC attached in both. Use:

```
--dampening_factor 0.1
--recurrent_dampening_factor 0.1
--voltage_gradient_dampening 0.0
--global_clipnorm 0.0
--debug_gradients
```

Record pre-clip global norms, per-variable gradient norms, nonfinite or loss-scale- skipped updates, total and component losses, firing rates and near-threshold voltage fractions, validation loss, wall-clock time and

memory. Then add clipping only as a safety rail: pick a threshold from a high percentile of the observed norms, so that it catches rare upper-tail updates rather than nearly every update.

Keep reset detachment if the pilot shows finite gradients without persistent clipping, equal or better validation convergence, and no pathological firing or voltage behavior. Restore the attached reset if detachment causes persistent norm growth, frequent nonfinite or skipped updates, or worse matched validation performance.

Do not test detached amplitudes near 0.175 on the full sequence, and never use detached 1.0. Do not use voltage self-loop dampening as the first response to instability (Section 3.6).

11. The exact membrane propagator (since 2026-09-25)

Commit [b0f4cbf](#) replaced the Euler step with a generalized step whose constants come from `glif_propagators.membrane_coefficients`:

$$v_{t+1} = \alpha v_t + \sum_b (\mathcal{A}_b y_{t,b} + \mathcal{B}_b x_{t,b}) + \sum_j D_j a_{t,j} + c_R \tilde{z}_t^R + s \tilde{z}_t^A, \quad a_{t+1,j} = \rho_j a_{t,j} + \hat{A}_j \tilde{z}_t^A,$$

where $\tilde{z}_t^R$ and $\tilde{z}_t^A$ are z_t with the reset and ASC detach flags applied. The two spike effects are kept as separate constants precisely so that the flags can gate them separately:

Constant	Euler (<code>scheme="euler"</code>)	Exact (<code>scheme="exact"</code>)
$\mathcal{A}_b, D_j$	$\kappa = (1 - \alpha)/g$	exact convolution integrals $\varphi_1(\cdot)/C$
$\mathcal{B}_b$	0	$\varphi_2(\cdot)/C$
c_R (<code>reset_coeff</code>)	-1	$-\alpha$
s (<code>asc_spike_factor</code>)	0	$\sum_j D_j A_j$
$\hat{A}_j$ (<code>asc_amps</code>)	A_j	$\rho_j A_j$

Under the exact scheme, a spike at grid point t resets the membrane and injects its ASC at t . The reset therefore decays across the step ($c_R = -\alpha$), and the new ASC drives the membrane within the same step (s). The Euler scheme reproduces the historical step exactly, so all experiments above correspond to `scheme="euler"`.

11.1 Jacobian under the exact scheme

$$\frac{\partial v_{t+1}}{\partial v_t} = \alpha + (\varepsilon_R c_R + \varepsilon_A s) g_t = \alpha - \varepsilon_R \alpha g_t + \varepsilon_A s g_t, \quad \frac{\partial a_{t+1,j}}{\partial v_t} = \varepsilon_A \rho_j A_j g_t, \quad \frac{\partial v_{t+1}}{\partial a_{t,j}} = D_j.$$

Two things change relative to Section 3.2:

1. **The attached-reset cancellation is weaker by a factor α .** The attached self-loop is $\alpha(1 - g_t)$ instead of $\alpha - g_t$. The attached spike gain becomes

$$G_{\text{attached}} = \frac{\bar{g}}{1 - \alpha + \alpha \bar{g}},$$

which equals exactly 1 at $\bar{g} = 1$ and is 0.64 at $\gamma = 0.1$ (median α). It still saturates, so the qualitative stability argument for attached resets carries over.

2. The ASC flag now also acts on the same-step voltage. With the ASC path attached, $s g_t$ enters the self-loop directly. In the 203k network s is negative for 91.0% of neurons (5th / 50th / 95th percentile: $-0.098 / -0.056 / +0.067$, i.e. about -0.06α at the median). With a detached reset and attached ASC, the self-loop becomes $\alpha + s g_t$, a small extra negative-feedback term of the same sign as the reset term but roughly 16 times smaller. The detached spike gain becomes approximately $\bar{g}/(1 - \alpha - s\bar{g})$, slightly below $\bar{g}/(1 - \alpha)$ for most neurons.

The detached-reset analysis of Sections 3.3--3.4 is otherwise unchanged: the spike gain still grows linearly with γ , so a critical amplitude still exists.

With `-hard_reset`, the current code adds $\text{sg}(z_t) (\alpha(V_{\text{reset}} - v_t) - c_R)$ to v_{t+1} . The spike keeps the soft reset's z -gradient, and the voltage derivative gains $-\alpha z_t$, so $\partial v_{t+1}/\partial v_t = \alpha(1 - z_t) + \dots$ on spike steps. The refractory clamp hides this unless refractoriness ends within the step. No experiment here used the hard reset.

11.2 What this means for the recommendation

These are predictions, not measurements. The exact scheme slightly weakens the attached-reset cancellation, adds a small same-step negative-feedback term through the attached ASC path, and changes the PSC and ASC drive coefficients. None of these alter the mechanism that makes detached resets need a small γ , but the stable boundary γ^* and the detached-vs-attached ranking may shift. The single-update sequence-500 gradient diagnostics (Section 9.8) and the sequence-500 comparison (Section 9.9) should be repeated under `scheme="exact"` before the recommendation is treated as valid for the current code.

Primary literature

- Eshraghian et al. (2023), [Training Spiking Neural Networks Using Lessons From Deep Learning](#).
- Zenke and Vogels (2021), [The Remarkable Robustness of Surrogate Gradient Learning for Instilling Complex Function in Spiking Neural Networks](#).
- Gygax and Zenke (2025), [Elucidating the Theoretical Underpinnings of Surrogate Gradient Learning in Spiking Neural Networks](#).
- Bellec et al. (2018), [Long short-term memory and learning-to-learn in networks of spiking neurons](#), NeurIPS.

Further sources are listed in the three literature documents in this folder.